YOHPAL LIVE CREATOR STUDIO RELEASE 1.1A Disruptive Video Creation Foundation — Full Implementation Codebase

Daniel Macharia <danielmacharia43@gmail.com> Tue, Jul 14, 2026 at 11:30 AM
To: daniel@ics.ac.ke

YOHPAL LIVE CREATOR STUDIO RELEASE 1.1A

Disruptive Video Creation Foundation — Full Implementation Codebase

This release establishes the launch foundation for:

- YohPal Camera with multi-clip recording.
- Timeline trim, split, delete and reorder.
- Music and voice-over tracks.
- Time-synchronized captions.
- Persistent drafts and autosave.
- Resumable uploads with pause, retry and cancel.
- Enforced publication gates.
- Timestamped Jobs, Hustle, Market and Course objects.
- Integration points for existing YohPal Brain captions, hooks, hashtags, thumbnail ideas, viral scores and trim recommendations.

The implementation is deliberately modular so YohPal can activate features progressively through flags rather than delaying launch until every advanced capability is complete.

---

1. Recommended File Structure

```
apps/mobile_flutter/
├── lib/
│   ├── core/
│   │   ├── config/
│   │   │   └── creator_studio_flags.dart
│   │   └── result/
│   │       └── app_result.dart
│   │
│   └── features/
│       └── creator_studio/
│           ├── creator_studio.dart
│           ├── domain/
│           │   ├── creator_objective.dart
│           │   ├── creator_project.dart
│           │   ├── creator_project_status.dart
│           │   ├── video_clip_edit.dart
│           │   ├── audio_track_edit.dart
│           │   ├── caption_segment.dart
│           │   ├── timeline_object.dart
│           │   ├── publication_gate.dart
```

```
│           │   └── brain_recommendations.dart
│           ├── data/
│           │   ├── creator_project_repository.dart
│           │   ├── hive_creator_project_repository.dart
│           │   ├── upload_repository.dart
│           │   ├── firebase_upload_repository.dart
│           │   ├── ecosystem_object_repository.dart
│           │   └── firebase_ecosystem_object_repository.dart
│           ├── services/
│           │   ├── creator_camera_service.dart
│           │   ├── creator_render_service.dart
│           │   ├── ffmpeg_command_builder.dart
│           │   ├── caption_generation_service.dart
│           │   ├── voice_over_service.dart
│           │   ├── creator_brain_service.dart
│           │   ├── creator_moderation_service.dart
│           │   └── creator_upload_coordinator.dart
│           ├── controllers/
│           │   ├── creator_project_controller.dart
│           │   ├── creator_camera_controller.dart
│           │   ├── creator_editor_controller.dart
│           │   └── creator_publish_controller.dart
│           └── presentation/
│               ├── creator_studio_home_screen.dart
│               ├── creator_camera_screen.dart
│               ├── creator_editor_screen.dart
│               ├── creator_caption_screen.dart
│               ├── creator_object_picker_screen.dart
│               ├── creator_publish_screen.dart
│               └── widgets/
│                   ├── timeline_strip.dart
│                   ├── clip_tile.dart
│                   ├── caption_overlay.dart
│                   ├── publication_gate_panel.dart
│                   └── upload_progress_card.dart
│
├── test/
│   └── features/
│       └── creator_studio/
│           ├── creator_project_test.dart
│           ├── publication_gate_test.dart
│           ├── ffmpeg_command_builder_test.dart
│           ├── creator_editor_controller_test.dart
│           └── creator_upload_coordinator_test.dart
│
backend/firebase_functions/
├── src/
│   └── creator-studio/
│       ├── creator-studio.routes.ts
│       ├── create-upload-session.ts
│       ├── finalize-video-upload.ts
│       ├── publish-creator-video.ts
│       ├── trusted-object-resolver.ts
│       ├── publication-gate.ts
│       └── creator-studio.types.ts
│
firestore/
├── firestore.rules
└── firestore.indexes.json
```

---

2. Flutter Dependencies

Add the following to `apps/mobile_flutter/pubspec.yaml`.

```
dependencies:
  flutter:
    sdk: flutter

  camera: ^0.11.0
  video_player: ^2.9.0
  image_picker: ^1.1.0
  path_provider: ^2.1.0
  permission_handler: ^11.0.0
  uuid: ^4.0.0
  equatable: ^2.0.0

  hive_ce: ^2.8.0
  hive_ce_flutter: ^2.1.0

  firebase_auth: ^5.0.0
  firebase_core: ^3.0.0
  firebase_storage: ^12.0.0
  cloud_firestore: ^5.0.0
  cloud_functions: ^5.0.0
  firebase_crashlytics: ^4.0.0
  firebase_analytics: ^11.0.0

dev_dependencies:
  flutter_test:
    sdk: flutter

  fake_async: ^1.3.0
  mocktail: ^1.0.0
  flutter_lints: ^5.0.0
```

Use the versions already approved in the repository lockfile where they differ.

---

3. Feature Flags

`lib/core/config/creator_studio_flags.dart`

```
final class CreatorStudioFlags {
  const CreatorStudioFlags._();

  static const bool enabled = bool.fromEnvironment(
    'CREATOR_STUDIO_ENABLED',
    defaultValue: true,
  );

  static const bool cameraEnabled = bool.fromEnvironment(
    'CREATOR_CAMERA_ENABLED',
    defaultValue: true,
  );

  static const bool multiClipEnabled = bool.fromEnvironment(
    'CREATOR_MULTI_CLIP_ENABLED',
    defaultValue: true,
  );

  static const bool captionsEnabled = bool.fromEnvironment(
    'CREATOR_CAPTIONS_ENABLED',
    defaultValue: true,
```

```
  );

  static const bool musicEnabled = bool.fromEnvironment(
    'CREATOR_MUSIC_ENABLED',
    defaultValue: true,
  );

  static const bool voiceOverEnabled = bool.fromEnvironment(
    'CREATOR_VOICE_OVER_ENABLED',
    defaultValue: true,
  );

  static const bool interactiveObjectsEnabled = bool.fromEnvironment(
    'CREATOR_OBJECTS_ENABLED',
    defaultValue: true,
  );

  static const bool aiRecommendationsEnabled = bool.fromEnvironment(
    'CREATOR_AI_ENABLED',
    defaultValue: true,
  );

  static const bool aiPublicationGateRequired = bool.fromEnvironment(
    'CREATOR_AI_GATE_REQUIRED',
    defaultValue: false,
  );
}
```

---

4. Shared Result Type

lib/core/result/app_result.dart

```
sealed class AppResult<T> {
  const AppResult();
}

final class AppSuccess<T> extends AppResult<T> {
  const AppSuccess(this.value);

  final T value;
}

final class AppFailure<T> extends AppResult<T> {
  const AppFailure({
    required this.message,
    this.error,
    this.stackTrace,
  });

  final String message;
  final Object? error;
  final StackTrace? stackTrace;
}
```

---

5. Domain Models

creator_objective.dart

```
enum CreatorObjective {
  entertain,
  growAudience,
  sellProduct,
  advertiseJob,
  promoteHustle,
  teach,
  promoteCourse,
  announceEvent,
  promoteBusiness,
}
```

creator_project_status.dart

```
enum CreatorProjectStatus {
  draft,
  recording,
  editing,
  rendering,
  rendered,
  uploadQueued,
  uploading,
  uploaded,
  processing,
  moderationPending,
  readyToPublish,
  published,
  failed,
}
```

video_clip_edit.dart

```
import 'package:equatable/equatable.dart';

final class VideoClipEdit extends Equatable {
  const VideoClipEdit({
    required this.id,
    required this.sourcePath,
    required this.originalDurationMs,
    required this.trimStartMs,
    required this.trimEndMs,
    this.speed = 1,
    this.volume = 1,
    this.rotationDegrees = 0,
    this.muted = false,
  });

  final String id;
  final String sourcePath;
  final int originalDurationMs;
  final int trimStartMs;
  final int trimEndMs;
  final double speed;
  final double volume;
  final int rotationDegrees;
  final bool muted;

  int get effectiveDurationMs {
    final trimmed = trimEndMs - trimStartMs;

    if (trimmed <= 0 || speed <= 0) {
```

```
    return 0;
  }

  return (trimmed / speed).round();
}

VideoClipEdit copyWith({
  String? id,
  String? sourcePath,
  int? originalDurationMs,
  int? trimStartMs,
  int? trimEndMs,
  double? speed,
  double? volume,
  int? rotationDegrees,
  bool? muted,
}) {
  return VideoClipEdit(
    id: id ?? this.id,
    sourcePath: sourcePath ?? this.sourcePath,
    originalDurationMs:
        originalDurationMs ?? this.originalDurationMs,
    trimStartMs: trimStartMs ?? this.trimStartMs,
    trimEndMs: trimEndMs ?? this.trimEndMs,
    speed: speed ?? this.speed,
    volume: volume ?? this.volume,
    rotationDegrees: rotationDegrees ?? this.rotationDegrees,
    muted: muted ?? this.muted,
  );
}

Map<String, dynamic> toJson() => {
      'id': id,
      'sourcePath': sourcePath,
      'originalDurationMs': originalDurationMs,
      'trimStartMs': trimStartMs,
      'trimEndMs': trimEndMs,
      'speed': speed,
      'volume': volume,
      'rotationDegrees': rotationDegrees,
      'muted': muted,
    };

factory VideoClipEdit.fromJson(Map<String, dynamic> json) {
  return VideoClipEdit(
    id: json['id'] as String,
    sourcePath: json['sourcePath'] as String,
    originalDurationMs: json['originalDurationMs'] as int,
    trimStartMs: json['trimStartMs'] as int,
    trimEndMs: json['trimEndMs'] as int,
    speed: (json['speed'] as num?)?.toDouble() ?? 1,
    volume: (json['volume'] as num?)?.toDouble() ?? 1,
    rotationDegrees: json['rotationDegrees'] as int? ?? 0,
    muted: json['muted'] as bool? ?? false,
  );
}

@override
List<Object?> get props => [
      id,
      sourcePath,
      originalDurationMs,
      trimStartMs,
```

```
        trimEndMs,
        speed,
        volume,
        rotationDegrees,
        muted,
      ];
}
```

audio_track_edit.dart

```
enum AudioTrackType {
  music,
  voiceOver,
  original,
  soundEffect,
}

final class AudioTrackEdit {
  const AudioTrackEdit({
    required this.id,
    required this.type,
    required this.sourcePath,
    required this.startAtMs,
    required this.trimStartMs,
    required this.trimEndMs,
    this.volume = 1,
    this.fadeInMs = 0,
    this.fadeOutMs = 0,
  });

  final String id;
  final AudioTrackType type;
  final String sourcePath;
  final int startAtMs;
  final int trimStartMs;
  final int trimEndMs;
  final double volume;
  final int fadeInMs;
  final int fadeOutMs;

  Map<String, dynamic> toJson() => {
        'id': id,
        'type': type.name,
        'sourcePath': sourcePath,
        'startAtMs': startAtMs,
        'trimStartMs': trimStartMs,
        'trimEndMs': trimEndMs,
        'volume': volume,
        'fadeInMs': fadeInMs,
        'fadeOutMs': fadeOutMs,
      };

  factory AudioTrackEdit.fromJson(Map<String, dynamic> json) {
    return AudioTrackEdit(
      id: json['id'] as String,
      type: AudioTrackType.values.byName(
        json['type'] as String,
      ),
      sourcePath: json['sourcePath'] as String,
      startAtMs: json['startAtMs'] as int,
      trimStartMs: json['trimStartMs'] as int,
      trimEndMs: json['trimEndMs'] as int,
```

```
      volume: (json['volume'] as num?)?.toDouble() ?? 1,
      fadeInMs: json['fadeInMs'] as int? ?? 0,
      fadeOutMs: json['fadeOutMs'] as int? ?? 0,
    );
  }
}
```

caption_segment.dart

```
final class CaptionSegment {
  const CaptionSegment({
    required this.id,
    required this.text,
    required this.startMs,
    required this.endMs,
    this.language = 'en',
    this.confidence = 1,
  });

  final String id;
  final String text;
  final int startMs;
  final int endMs;
  final String language;
  final double confidence;

  CaptionSegment copyWith({
    String? text,
    int? startMs,
    int? endMs,
    String? language,
    double? confidence,
  }) {
    return CaptionSegment(
      id: id,
      text: text ?? this.text,
      startMs: startMs ?? this.startMs,
      endMs: endMs ?? this.endMs,
      language: language ?? this.language,
      confidence: confidence ?? this.confidence,
    );
  }

  Map<String, dynamic> toJson() => {
        'id': id,
        'text': text,
        'startMs': startMs,
        'endMs': endMs,
        'language': language,
        'confidence': confidence,
      };

  factory CaptionSegment.fromJson(Map<String, dynamic> json) {
    return CaptionSegment(
      id: json['id'] as String,
      text: json['text'] as String,
      startMs: json['startMs'] as int,
      endMs: json['endMs'] as int,
      language: json['language'] as String? ?? 'en',
      confidence: (json['confidence'] as num?)?.toDouble() ?? 1,
    );
  }
```

```
}
```

timeline_object.dart

```
enum TimelineObjectType {
  job,
  hustle,
  marketProduct,
  course,
  business,
  walletOffer,
  aiAsk,
}

final class TimelineObject {
  const TimelineObject({
    required this.id,
    required this.type,
    required this.referenceId,
    required this.title,
    required this.appearAtMs,
    required this.disappearAtMs,
    required this.deepLink,
    this.thumbnailUrl,
  });

  final String id;
  final TimelineObjectType type;
  final String referenceId;
  final String title;
  final int appearAtMs;
  final int disappearAtMs;
  final String deepLink;
  final String? thumbnailUrl;

  bool isVisibleAt(int positionMs) {
    return positionMs >= appearAtMs &&
        positionMs <= disappearAtMs;
  }

  Map<String, dynamic> toJson() => {
        'id': id,
        'type': type.name,
        'referenceId': referenceId,
        'title': title,
        'appearAtMs': appearAtMs,
        'disappearAtMs': disappearAtMs,
        'deepLink': deepLink,
        'thumbnailUrl': thumbnailUrl,
      };

  factory TimelineObject.fromJson(Map<String, dynamic> json) {
    return TimelineObject(
      id: json['id'] as String,
      type: TimelineObjectType.values.byName(
        json['type'] as String,
      ),
      referenceId: json['referenceId'] as String,
      title: json['title'] as String,
      appearAtMs: json['appearAtMs'] as int,
      disappearAtMs: json['disappearAtMs'] as int,
      deepLink: json['deepLink'] as String,
```

```
      thumbnailUrl: json['thumbnailUrl'] as String?,
    );
  }
}
```

brain_recommendations.dart

```
final class BrainRecommendations {
  const BrainRecommendations({
    required this.captionSuggestions,
    required this.hookSuggestions,
    required this.hashtags,
    required this.thumbnailIdeas,
    required this.trimSuggestions,
    required this.viralScore,
    required this.explanation,
  });

  final List<String> captionSuggestions;
  final List<String> hookSuggestions;
  final List<String> hashtags;
  final List<String> thumbnailIdeas;
  final List<BrainTrimSuggestion> trimSuggestions;
  final double viralScore;
  final String explanation;

  factory BrainRecommendations.fromJson(
    Map<String, dynamic> json,
  ) {
    return BrainRecommendations(
      captionSuggestions: List<String>.from(
        json['captionSuggestions'] as List? ?? const [],
      ),
      hookSuggestions: List<String>.from(
        json['hookSuggestions'] as List? ?? const [],
      ),
      hashtags: List<String>.from(
        json['hashtags'] as List? ?? const [],
      ),
      thumbnailIdeas: List<String>.from(
        json['thumbnailIdeas'] as List? ?? const [],
      ),
      trimSuggestions:
          (json['trimSuggestions'] as List? ?? const [])
              .map(
                (item) => BrainTrimSuggestion.fromJson(
                  Map<String, dynamic>.from(item as Map),
                ),
              )
              .toList(growable: false),
      viralScore:
          (json['viralScore'] as num?)?.toDouble() ?? 0,
      explanation: json['explanation'] as String? ?? '',
    );
  }
}

final class BrainTrimSuggestion {
  const BrainTrimSuggestion({
    required this.clipId,
    required this.startMs,
    required this.endMs,
```

```
    required this.reason,
    required this.confidence,
  });

  final String clipId;
  final int startMs;
  final int endMs;
  final String reason;
  final double confidence;

  factory BrainTrimSuggestion.fromJson(
    Map<String, dynamic> json,
  ) {
    return BrainTrimSuggestion(
      clipId: json['clipId'] as String,
      startMs: json['startMs'] as int,
      endMs: json['endMs'] as int,
      reason: json['reason'] as String? ?? '',
      confidence:
          (json['confidence'] as num?)?.toDouble() ?? 0,
    );
  }
}
```

creator_project.dart

```
import 'audio_track_edit.dart';
import 'brain_recommendations.dart';
import 'caption_segment.dart';
import 'creator_objective.dart';
import 'creator_project_status.dart';
import 'timeline_object.dart';
import 'video_clip_edit.dart';

final class CreatorProject {
  const CreatorProject({
    required this.id,
    required this.ownerId,
    required this.title,
    required this.objective,
    required this.status,
    required this.clips,
    required this.audioTracks,
    required this.captions,
    required this.timelineObjects,
    required this.createdAt,
    required this.updatedAt,
    this.description = '',
    this.coverPath,
    this.renderedPath,
    this.uploadId,
    this.remoteVideoUrl,
    this.brainRecommendations,
    this.selectedCaption,
    this.selectedHook,
    this.selectedThumbnailIdea,
    this.hashtags = const [],
  });

  final String id;
  final String ownerId;
  final String title;
```

```
final String description;
final CreatorObjective objective;
final CreatorProjectStatus status;

final List<VideoClipEdit> clips;
final List<AudioTrackEdit> audioTracks;
final List<CaptionSegment> captions;
final List<TimelineObject> timelineObjects;

final DateTime createdAt;
final DateTime updatedAt;

final String? coverPath;
final String? renderedPath;
final String? uploadId;
final String? remoteVideoUrl;

final BrainRecommendations? brainRecommendations;
final String? selectedCaption;
final String? selectedHook;
final String? selectedThumbnailIdea;
final List<String> hashtags;

int get totalDurationMs {
  return clips.fold<int>(
    0,
    (total, clip) => total + clip.effectiveDurationMs,
  );
}

CreatorProject copyWith({
  String? title,
  String? description,
  CreatorObjective? objective,
  CreatorProjectStatus? status,
  List<VideoClipEdit>? clips,
  List<AudioTrackEdit>? audioTracks,
  List<CaptionSegment>? captions,
  List<TimelineObject>? timelineObjects,
  DateTime? updatedAt,
  String? coverPath,
  String? renderedPath,
  String? uploadId,
  String? remoteVideoUrl,
  BrainRecommendations? brainRecommendations,
  String? selectedCaption,
  String? selectedHook,
  String? selectedThumbnailIdea,
  List<String>? hashtags,
}) {
  return CreatorProject(
    id: id,
    ownerId: ownerId,
    title: title ?? this.title,
    description: description ?? this.description,
    objective: objective ?? this.objective,
    status: status ?? this.status,
    clips: clips ?? this.clips,
    audioTracks: audioTracks ?? this.audioTracks,
    captions: captions ?? this.captions,
    timelineObjects: timelineObjects ?? this.timelineObjects,
    createdAt: createdAt,
    updatedAt: updatedAt ?? DateTime.now(),
```

```
    coverPath: coverPath ?? this.coverPath,
    renderedPath: renderedPath ?? this.renderedPath,
    uploadId: uploadId ?? this.uploadId,
    remoteVideoUrl: remoteVideoUrl ?? this.remoteVideoUrl,
    brainRecommendations:
        brainRecommendations ?? this.brainRecommendations,
    selectedCaption:
        selectedCaption ?? this.selectedCaption,
    selectedHook: selectedHook ?? this.selectedHook,
    selectedThumbnailIdea:
        selectedThumbnailIdea ?? this.selectedThumbnailIdea,
    hashtags: hashtags ?? this.hashtags,
  );
}

Map<String, dynamic> toJson() => {
      'id': id,
      'ownerId': ownerId,
      'title': title,
      'description': description,
      'objective': objective.name,
      'status': status.name,
      'clips': clips.map((item) => item.toJson()).toList(),
      'audioTracks':
          audioTracks.map((item) => item.toJson()).toList(),
      'captions':
          captions.map((item) => item.toJson()).toList(),
      'timelineObjects':
          timelineObjects.map((item) => item.toJson()).toList(),
      'createdAt': createdAt.toIso8601String(),
      'updatedAt': updatedAt.toIso8601String(),
      'coverPath': coverPath,
      'renderedPath': renderedPath,
      'uploadId': uploadId,
      'remoteVideoUrl': remoteVideoUrl,
      'selectedCaption': selectedCaption,
      'selectedHook': selectedHook,
      'selectedThumbnailIdea': selectedThumbnailIdea,
      'hashtags': hashtags,
    };

factory CreatorProject.fromJson(Map<String, dynamic> json) {
  return CreatorProject(
    id: json['id'] as String,
    ownerId: json['ownerId'] as String,
    title: json['title'] as String,
    description: json['description'] as String? ?? '',
    objective: CreatorObjective.values.byName(
      json['objective'] as String,
    ),
    status: CreatorProjectStatus.values.byName(
      json['status'] as String,
    ),
    clips: (json['clips'] as List? ?? const [])
        .map(
          (item) => VideoClipEdit.fromJson(
            Map<String, dynamic>.from(item as Map),
          ),
        )
        .toList(),
    audioTracks:
        (json['audioTracks'] as List? ?? const [])
            .map(
```

```
                (item) => AudioTrackEdit.fromJson(
                  Map<String, dynamic>.from(item as Map),
                ),
              )
              .toList(),
      captions: (json['captions'] as List? ?? const [])
          .map(
            (item) => CaptionSegment.fromJson(
              Map<String, dynamic>.from(item as Map),
            ),
          )
          .toList(),
      timelineObjects:
          (json['timelineObjects'] as List? ?? const [])
              .map(
                (item) => TimelineObject.fromJson(
                  Map<String, dynamic>.from(item as Map),
                ),
              )
              .toList(),
      createdAt: DateTime.parse(json['createdAt'] as String),
      updatedAt: DateTime.parse(json['updatedAt'] as String),
      coverPath: json['coverPath'] as String?,
      renderedPath: json['renderedPath'] as String?,
      uploadId: json['uploadId'] as String?,
      remoteVideoUrl: json['remoteVideoUrl'] as String?,
      selectedCaption: json['selectedCaption'] as String?,
      selectedHook: json['selectedHook'] as String?,
      selectedThumbnailIdea:
          json['selectedThumbnailIdea'] as String?,
      hashtags:
          List<String>.from(json['hashtags'] as List? ?? const []),
    );
  }
}
```

---

6. Enforced Publication Gate

publication_gate.dart

```
final class PublicationGateInput {
  const PublicationGateInput({
    required this.hasRenderedVideo,
    required this.uploadComplete,
    required this.moderationPassed,
    required this.copyrightPassed,
    required this.metadataComplete,
    required this.captionReviewComplete,
    required this.validTimelineObjects,
    required this.aiChecklistRequired,
    required this.aiChecklistComplete,
  });

  final bool hasRenderedVideo;
  final bool uploadComplete;
  final bool moderationPassed;
  final bool copyrightPassed;
  final bool metadataComplete;
  final bool captionReviewComplete;
  final bool validTimelineObjects;
```

```
  final bool aiChecklistRequired;
  final bool aiChecklistComplete;
}

final class PublicationGateResult {
  const PublicationGateResult({
    required this.allowed,
    required this.blockers,
  });

  final bool allowed;
  final List<String> blockers;
}

final class VideoPublicationGate {
  const VideoPublicationGate();

  PublicationGateResult evaluate(
    PublicationGateInput input,
  ) {
    final blockers = <String>[];

    if (!input.hasRenderedVideo) {
      blockers.add('Video rendering is incomplete.');
    }

    if (!input.uploadComplete) {
      blockers.add('Video upload is incomplete.');
    }

    if (!input.moderationPassed) {
      blockers.add('Safety moderation has not passed.');
    }

    if (!input.copyrightPassed) {
      blockers.add('Copyright verification has not passed.');
    }

    if (!input.metadataComplete) {
      blockers.add('Caption, title or visibility is incomplete.');
    }

    if (!input.captionReviewComplete) {
      blockers.add('Generated captions require review.');
    }

    if (!input.validTimelineObjects) {
      blockers.add(
        'One or more interactive objects are invalid.',
      );
    }

    if (input.aiChecklistRequired &&
        !input.aiChecklistComplete) {
      blockers.add(
        'AI publishing checklist is incomplete.',
      );
    }

    return PublicationGateResult(
      allowed: blockers.isEmpty,
      blockers: blockers,
    );
```

```
  }
}
```

---

7. Project Persistence

creator_project_repository.dart

```
import '../domain/creator_project.dart';

abstract interface class CreatorProjectRepository {
  Future<void> initialize();

  Future<void> save(CreatorProject project);

  Future<CreatorProject?> findById(String projectId);

  Future<List<CreatorProject>> findByOwner(String ownerId);

  Future<void> delete(String projectId);

  Stream<List<CreatorProject>> watchByOwner(String ownerId);
}
```

hive_creator_project_repository.dart

```
import 'dart:async';
import 'dart:convert';

import 'package:hive_ce/hive.dart';

import '../domain/creator_project.dart';
import 'creator_project_repository.dart';

final class HiveCreatorProjectRepository
    implements CreatorProjectRepository {
  static const String _boxName =
      'yohpal_creator_studio_projects_v1';

  Box<String>? _box;

  final StreamController<void> _changes =
      StreamController<void>.broadcast();

  @override
  Future<void> initialize() async {
    _box ??= await Hive.openBox<String>(_boxName);
  }

  @override
  Future<void> save(CreatorProject project) async {
    await initialize();

    await _box!.put(
      project.id,
      jsonEncode(project.toJson()),
    );

    _changes.add(null);
  }
```

```
  @override
  Future<CreatorProject?> findById(String projectId) async {
    await initialize();

    final raw = _box!.get(projectId);

    if (raw == null) {
      return null;
    }

    return CreatorProject.fromJson(
      Map<String, dynamic>.from(
        jsonDecode(raw) as Map,
      ),
    );
  }

  @override
  Future<List<CreatorProject>> findByOwner(
    String ownerId,
  ) async {
    await initialize();

    final projects = <CreatorProject>[];

    for (final raw in _box!.values) {
      final project = CreatorProject.fromJson(
        Map<String, dynamic>.from(
          jsonDecode(raw) as Map,
        ),
      );

      if (project.ownerId == ownerId) {
        projects.add(project);
      }
    }

    projects.sort(
      (a, b) => b.updatedAt.compareTo(a.updatedAt),
    );

    return projects;
  }

  @override
  Future<void> delete(String projectId) async {
    await initialize();
    await _box!.delete(projectId);
    _changes.add(null);
  }

  @override
  Stream<List<CreatorProject>> watchByOwner(
    String ownerId,
  ) async* {
    yield await findByOwner(ownerId);

    await for (final _ in _changes.stream) {
      yield await findByOwner(ownerId);
    }
  }
}
```

8. Camera Service

creator_camera_service.dart

```
import 'package:camera/camera.dart';

abstract interface class CreatorCameraService {
  CameraController? get controller;

  Future<List<CameraDescription>> discover();

  Future<void> initialize({
    required CameraDescription camera,
    ResolutionPreset resolution,
  });

  Future<String> startRecording();

  Future<String> stopRecording();

  Future<void> pauseRecording();

  Future<void> resumeRecording();

  Future<void> switchCamera();

  Future<void> setFlashMode(FlashMode mode);

  Future<void> dispose();
}

final class DefaultCreatorCameraService
    implements CreatorCameraService {
  DefaultCreatorCameraService();

  List<CameraDescription> _cameras = const [];
  CameraController? _controller;
  int _selectedIndex = 0;

  @override
  CameraController? get controller => _controller;

  @override
  Future<List<CameraDescription>> discover() async {
    _cameras = await availableCameras();
    return _cameras;
  }

  @override
  Future<void> initialize({
    required CameraDescription camera,
    ResolutionPreset resolution =
        ResolutionPreset.high,
  }) async {
    await _controller?.dispose();

    _selectedIndex = _cameras.indexOf(camera);

    _controller = CameraController(
      camera,
```

```
    resolution,
    enableAudio: true,
    imageFormatGroup: ImageFormatGroup.yuv420,
  );

  await _controller!.initialize();
}

@override
Future<String> startRecording() async {
  final camera = _controller;

  if (camera == null || !camera.value.isInitialized) {
    throw StateError('Camera is not initialized.');
  }

  if (camera.value.isRecordingVideo) {
    throw StateError('Recording already active.');
  }

  await camera.startVideoRecording();

  return DateTime.now().millisecondsSinceEpoch.toString();
}

@override
Future<String> stopRecording() async {
  final camera = _controller;

  if (camera == null || !camera.value.isRecordingVideo) {
    throw StateError('No active recording.');
  }

  final file = await camera.stopVideoRecording();

  return file.path;
}

@override
Future<void> pauseRecording() async {
  final camera = _controller;

  if (camera == null || !camera.value.isRecordingVideo) {
    return;
  }

  await camera.pauseVideoRecording();
}

@override
Future<void> resumeRecording() async {
  final camera = _controller;

  if (camera == null || !camera.value.isRecordingPaused) {
    return;
  }

  await camera.resumeVideoRecording();
}

@override
Future<void> switchCamera() async {
  if (_cameras.length < 2) {
```

```
      return;
    }

    _selectedIndex =
        (_selectedIndex + 1) % _cameras.length;

    await initialize(
      camera: _cameras[_selectedIndex],
    );
  }

  @override
  Future<void> setFlashMode(FlashMode mode) async {
    await _controller?.setFlashMode(mode);
  }

  @override
  Future<void> dispose() async {
    await _controller?.dispose();
    _controller = null;
  }
}
```

---

9. FFmpeg Render Contract

The mobile project should use the repository-approved FFmpeg package or native rendering worker. The command builder below remains independent of the chosen FFmpeg binding.

`ffmpeg_command_builder.dart`

```
import '../domain/audio_track_edit.dart';
import '../domain/creator_project.dart';
import '../domain/video_clip_edit.dart';

final class FfmpegRenderPlan {
  const FfmpegRenderPlan({
    required this.arguments,
    required this.outputPath,
  });

  final List<String> arguments;
  final String outputPath;
}

final class FfmpegCommandBuilder {
  const FfmpegCommandBuilder();

  FfmpegRenderPlan build({
    required CreatorProject project,
    required String outputPath,
  }) {
    if (project.clips.isEmpty) {
      throw ArgumentError('Project must contain at least one clip.');
    }

    final arguments = <String>[];
    final filterParts = <String>[];
    final videoLabels = <String>[];
    final audioLabels = <String>[];
```

```
for (var index = 0;
    index < project.clips.length;
    index++) {
  final clip = project.clips[index];

  arguments.addAll(['-i', clip.sourcePath]);

  final videoLabel = 'v$index';
  final audioLabel = 'a$index';

  filterParts.add(
    _videoFilter(
      clip: clip,
      inputIndex: index,
      outputLabel: videoLabel,
    ),
  );

  filterParts.add(
    _audioFilter(
      clip: clip,
      inputIndex: index,
      outputLabel: audioLabel,
    ),
  );

  videoLabels.add('[$videoLabel]');
  audioLabels.add('[$audioLabel]');
}

final concatVideoLabel = 'vconcat';
final concatAudioLabel = 'aconcat';

filterParts.add(
  '${videoLabels.join()}'
  '${audioLabels.join()}'
  'concat=n=${project.clips.length}:v=1:a=1'
  '[$concatVideoLabel][$concatAudioLabel]',
);

var currentAudioLabel = concatAudioLabel;

for (var index = 0;
    index < project.audioTracks.length;
    index++) {
  final track = project.audioTracks[index];

  arguments.addAll(['-i', track.sourcePath]);

  final inputIndex = project.clips.length + index;
  final prepared = 'extra_audio_$index';
  final mixed = 'mixed_audio_$index';

  filterParts.add(
    _extraAudioFilter(
      track: track,
      inputIndex: inputIndex,
      outputLabel: prepared,
    ),
  );

  filterParts.add(
    '[$currentAudioLabel][$prepared]'
```

```
      'amix=inputs=2:duration=first:dropout_transition=2'
      '[$mixed]',
    );

    currentAudioLabel = mixed;
  }

  arguments.addAll([
    '-filter_complex',
    filterParts.join(';'),
    '-map',
    '[$concatVideoLabel]',
    '-map',
    '[$currentAudioLabel]',
    '-c:v',
    'libx264',
    '-preset',
    'veryfast',
    '-crf',
    '22',
    '-pix_fmt',
    'yuv420p',
    '-c:a',
    'aac',
    '-b:a',
    '128k',
    '-movflags',
    '+faststart',
    '-shortest',
    '-y',
    outputPath,
  ]);

  return FfmpegRenderPlan(
    arguments: arguments,
    outputPath: outputPath,
  );
}

String _videoFilter({
  required VideoClipEdit clip,
  required int inputIndex,
  required String outputLabel,
}) {
  final start = clip.trimStartMs / 1000;
  final end = clip.trimEndMs / 1000;

  final filters = <String>[
    'trim=start=$start:end=$end',
    'setpts=(PTS-STARTPTS)/${clip.speed}',
    'scale=1080:1920:force_original_aspect_ratio=decrease',
    'pad=1080:1920:(ow-iw)/2:(oh-ih)/2:black',
    'setsar=1',
  ];

  switch (clip.rotationDegrees % 360) {
    case 90:
      filters.add('transpose=1');
    case 180:
      filters.add('hflip,vflip');
    case 270:
      filters.add('transpose=2');
  }
```

```
    return '[$inputIndex:v]${filters.join(',')}[$outputLabel]';
  }

  String _audioFilter({
    required VideoClipEdit clip,
    required int inputIndex,
    required String outputLabel,
  }) {
    final start = clip.trimStartMs / 1000;
    final end = clip.trimEndMs / 1000;
    final volume = clip.muted ? 0 : clip.volume;

    return '[$inputIndex:a]'
        'atrim=start=$start:end=$end,'
        'asetpts=PTS-STARTPTS,'
        'volume=$volume'
        '[$outputLabel]';
  }

  String _extraAudioFilter({
    required AudioTrackEdit track,
    required int inputIndex,
    required String outputLabel,
  }) {
    final start = track.trimStartMs / 1000;
    final end = track.trimEndMs / 1000;
    final delay = track.startAtMs;

    final filters = <String>[
      'atrim=start=$start:end=$end',
      'asetpts=PTS-STARTPTS',
      'volume=${track.volume}',
      'adelay=$delay|$delay',
    ];

    if (track.fadeInMs > 0) {
      filters.add(
        'afade=t=in:st=0:d=${track.fadeInMs / 1000}',
      );
    }

    if (track.fadeOutMs > 0) {
      final duration =
          (track.trimEndMs - track.trimStartMs) / 1000;

      final startFade =
          (duration - track.fadeOutMs / 1000).clamp(0, duration);

      filters.add(
        'afade=t=out:st=$startFade:d=${track.fadeOutMs / 1000}',
      );
    }

    return '[$inputIndex:a]${filters.join(',')}[$outputLabel]';
  }
}
```

creator_render_service.dart

```
import '../domain/creator_project.dart';
import 'ffmpeg_command_builder.dart';
```

```
abstract interface class CreatorRenderService {
  Stream<double> get progress;

  Future<String> render({
    required CreatorProject project,
    required String outputPath,
  });

  Future<void> cancel();
}

abstract interface class FfmpegExecutor {
  Future<int> execute(
    List<String> arguments, {
    required void Function(double progress) onProgress,
  });

  Future<void> cancel();
}

final class DefaultCreatorRenderService
    implements CreatorRenderService {
  DefaultCreatorRenderService({
    required FfmpegExecutor executor,
    FfmpegCommandBuilder commandBuilder =
        const FfmpegCommandBuilder(),
  })  : _executor = executor,
        _commandBuilder = commandBuilder;

  final FfmpegExecutor _executor;
  final FfmpegCommandBuilder _commandBuilder;

  final _progressController =
      StreamController<double>.broadcast();

  @override
  Stream<double> get progress => _progressController.stream;

  @override
  Future<String> render({
    required CreatorProject project,
    required String outputPath,
  }) async {
    final plan = _commandBuilder.build(
      project: project,
      outputPath: outputPath,
    );

    final exitCode = await _executor.execute(
      plan.arguments,
      onProgress: _progressController.add,
    );

    if (exitCode != 0) {
      throw StateError(
        'Video rendering failed with exit code $exitCode.',
      );
    }

    _progressController.add(1);

    return plan.outputPath;
```

```
  }

  @override
  Future<void> cancel() {
    return _executor.cancel();
  }
}
```

Add:

```
import 'dart:async';
```

at the top of `creator_render_service.dart`.

---

10. Caption Generation

`caption_generation_service.dart`

```
import '../domain/caption_segment.dart';

abstract interface class CaptionGenerationService {
  Future<List<CaptionSegment>> transcribe({
    required String mediaPath,
    required String language,
  });

  Future<List<CaptionSegment>> translate({
    required List<CaptionSegment> captions,
    required String targetLanguage,
  });
}

final class YohPalBrainCaptionGenerationService
    implements CaptionGenerationService {
  YohPalBrainCaptionGenerationService({
    required Uri baseUri,
    required Future<String> Function() accessTokenProvider,
    HttpClient? client,
  })  : _baseUri = baseUri,
        _accessTokenProvider = accessTokenProvider,
        _client = client ?? HttpClient();

  final Uri _baseUri;
  final Future<String> Function() _accessTokenProvider;
  final HttpClient _client;

  @override
  Future<List<CaptionSegment>> transcribe({
    required String mediaPath,
    required String language,
  }) async {
    final token = await _accessTokenProvider();

    final request = await _client.postUrl(
      _baseUri.resolve('/api/ai/creator/transcribe'),
    );

    request.headers
      ..set(HttpHeaders.authorizationHeader, 'Bearer $token')
      ..contentType = ContentType.json;
```

```
  request.write(
    jsonEncode({
      'mediaPath': mediaPath,
      'language': language,
      'operation': 'creator.caption.transcribe',
    }),
  );

  final response = await request.close();

  if (response.statusCode < 200 ||
      response.statusCode >= 300) {
    throw HttpException(
      'Caption generation failed: ${response.statusCode}',
    );
  }

  final body = await response.transform(utf8.decoder).join();
  final json = jsonDecode(body) as Map<String, dynamic>;

  return (json['segments'] as List? ?? const [])
      .map(
        (item) => CaptionSegment.fromJson(
          Map<String, dynamic>.from(item as Map),
        ),
      )
      .toList(growable: false);
}

@override
Future<List<CaptionSegment>> translate({
  required List<CaptionSegment> captions,
  required String targetLanguage,
}) async {
  final token = await _accessTokenProvider();

  final request = await _client.postUrl(
    _baseUri.resolve('/api/ai/creator/translate-captions'),
  );

  request.headers
    ..set(HttpHeaders.authorizationHeader, 'Bearer $token')
    ..contentType = ContentType.json;

  request.write(
    jsonEncode({
      'segments':
          captions.map((item) => item.toJson()).toList(),
      'targetLanguage': targetLanguage,
      'operation': 'creator.caption.translate',
    }),
  );

  final response = await request.close();

  if (response.statusCode < 200 ||
      response.statusCode >= 300) {
    throw HttpException(
      'Caption translation failed: ${response.statusCode}',
    );
  }
```

```
    final body = await response.transform(utf8.decoder).join();
    final json = jsonDecode(body) as Map<String, dynamic>;

    return (json['segments'] as List? ?? const [])
        .map(
          (item) => CaptionSegment.fromJson(
            Map<String, dynamic>.from(item as Map),
          ),
        )
        .toList(growable: false);
  }
}
```

Required imports:

```
import 'dart:convert';
import 'dart:io';
```

---

11. YohPal Brain Recommendation Service

`creator_brain_service.dart`

```
import 'dart:convert';
import 'dart:io';

import '../domain/brain_recommendations.dart';
import '../domain/creator_project.dart';

abstract interface class CreatorBrainService {
  Future<BrainRecommendations> analyse(
    CreatorProject project,
  );
}

final class HttpCreatorBrainService
    implements CreatorBrainService {
  HttpCreatorBrainService({
    required Uri baseUri,
    required Future<String> Function() accessTokenProvider,
    HttpClient? client,
  })  : _baseUri = baseUri,
        _accessTokenProvider = accessTokenProvider,
        _client = client ?? HttpClient();

  final Uri _baseUri;
  final Future<String> Function() _accessTokenProvider;
  final HttpClient _client;

  @override
  Future<BrainRecommendations> analyse(
    CreatorProject project,
  ) async {
    final token = await _accessTokenProvider();

    final request = await _client.postUrl(
      _baseUri.resolve('/api/ai/messages'),
    );

    request.headers
      ..set(HttpHeaders.authorizationHeader, 'Bearer $token')
```

```
      ..contentType = ContentType.json;

    request.write(
      jsonEncode({
        'productContext': {
          'product': 'yohpal-live',
          'module': 'creator-studio',
          'projectId': project.id,
          'creatorId': project.ownerId,
        },
        'operation': 'creator.production.analyse',
        'data': {
          'objective': project.objective.name,
          'durationMs': project.totalDurationMs,
          'clips': project.clips
              .map((clip) => clip.toJson())
              .toList(),
          'captions': project.captions
              .map((caption) => caption.toJson())
              .toList(),
          'objects': project.timelineObjects
              .map((object) => object.toJson())
              .toList(),
        },
      }),
    );

    final response = await request.close();

    if (response.statusCode < 200 ||
        response.statusCode >= 300) {
      throw HttpException(
        'YohPal Brain analysis failed: ${response.statusCode}',
      );
    }

    final payload =
        await response.transform(utf8.decoder).join();

    return BrainRecommendations.fromJson(
      jsonDecode(payload) as Map<String, dynamic>,
    );
  }
}
```

---

12. Resumable Upload Contract

upload_repository.dart

```
enum VideoUploadStatus {
  queued,
  uploading,
  paused,
  completed,
  failed,
  cancelled,
}

final class VideoUploadState {
  const VideoUploadState({
```

```
    required this.uploadId,
    required this.status,
    required this.progress,
    this.remoteUrl,
    this.errorMessage,
  });

  final String uploadId;
  final VideoUploadStatus status;
  final double progress;
  final String? remoteUrl;
  final String? errorMessage;
}

abstract interface class UploadRepository {
  Stream<VideoUploadState> watch(String uploadId);

  Future<String> start({
    required String projectId,
    required String ownerId,
    required String filePath,
  });

  Future<void> pause(String uploadId);

  Future<void> resume(String uploadId);

  Future<void> retry(String uploadId);

  Future<void> cancel(String uploadId);
}
```

firebase_upload_repository.dart

```
import 'dart:async';
import 'dart:io';

import 'package:firebase_storage/firebase_storage.dart';
import 'package:uuid/uuid.dart';

import 'upload_repository.dart';

final class FirebaseUploadRepository
    implements UploadRepository {
  FirebaseUploadRepository({
    FirebaseStorage? storage,
    Uuid uuid = const Uuid(),
  })  : _storage = storage ?? FirebaseStorage.instance,
        _uuid = uuid;

  final FirebaseStorage _storage;
  final Uuid _uuid;

  final Map<String, UploadTask> _tasks = {};
  final Map<String, _UploadDefinition> _definitions = {};
  final Map<String, StreamController<VideoUploadState>>
      _controllers = {};

  @override
  Future<String> start({
    required String projectId,
    required String ownerId,
```

```
  required String filePath,
}) async {
  final uploadId = _uuid.v4();

  _definitions[uploadId] = _UploadDefinition(
    projectId: projectId,
    ownerId: ownerId,
    filePath: filePath,
  );

  _controllers[uploadId] =
      StreamController<VideoUploadState>.broadcast();

  await _startTask(uploadId);

  return uploadId;
}

Future<void> _startTask(String uploadId) async {
  final definition = _definitions[uploadId];

  if (definition == null) {
    throw StateError('Upload definition not found.');
  }

  final file = File(definition.filePath);

  if (!await file.exists()) {
    throw FileSystemException(
      'Rendered file not found.',
      definition.filePath,
    );
  }

  final reference = _storage.ref(
    'creator-videos/'
    '${definition.ownerId}/'
    '${definition.projectId}/'
    '$uploadId.mp4',
  );

  final task = reference.putFile(
    file,
    SettableMetadata(
      contentType: 'video/mp4',
      customMetadata: {
        'uploadId': uploadId,
        'projectId': definition.projectId,
        'ownerId': definition.ownerId,
        'source': 'creator-studio-1.1a',
      },
    ),
  );

  _tasks[uploadId] = task;

  task.snapshotEvents.listen(
    (snapshot) {
      final progress = snapshot.totalBytes == 0
          ? 0.0
          : snapshot.bytesTransferred / snapshot.totalBytes;

      _emit(
```

```
        VideoUploadState(
          uploadId: uploadId,
          status: switch (snapshot.state) {
            TaskState.paused => VideoUploadStatus.paused,
            TaskState.success => VideoUploadStatus.completed,
            TaskState.canceled => VideoUploadStatus.cancelled,
            TaskState.error => VideoUploadStatus.failed,
            _ => VideoUploadStatus.uploading,
          },
          progress: progress,
        ),
      );
    },
    onError: (Object error) {
      _emit(
        VideoUploadState(
          uploadId: uploadId,
          status: VideoUploadStatus.failed,
          progress: 0,
          errorMessage: error.toString(),
        ),
      );
    },
    onDone: () async {
      try {
        final url = await reference.getDownloadURL();

        _emit(
          VideoUploadState(
            uploadId: uploadId,
            status: VideoUploadStatus.completed,
            progress: 1,
            remoteUrl: url,
          ),
        );
      } catch (error) {
        _emit(
          VideoUploadState(
            uploadId: uploadId,
            status: VideoUploadStatus.failed,
            progress: 1,
            errorMessage: error.toString(),
          ),
        );
      }
    },
  );
}

void _emit(VideoUploadState state) {
  final controller = _controllers[state.uploadId];

  if (controller == null || controller.isClosed) {
    return;
  }

  controller.add(state);
}

@override
Stream<VideoUploadState> watch(String uploadId) {
  final controller = _controllers[uploadId];
```

```
    if (controller == null) {
      throw StateError('Upload $uploadId is not registered.');
    }

    return controller.stream;
  }

  @override
  Future<void> pause(String uploadId) async {
    await _tasks[uploadId]?.pause();
  }

  @override
  Future<void> resume(String uploadId) async {
    await _tasks[uploadId]?.resume();
  }

  @override
  Future<void> retry(String uploadId) async {
    final old = _tasks.remove(uploadId);
    await old?.cancel();

    await _startTask(uploadId);
  }

  @override
  Future<void> cancel(String uploadId) async {
    await _tasks.remove(uploadId)?.cancel();

    _emit(
      VideoUploadState(
        uploadId: uploadId,
        status: VideoUploadStatus.cancelled,
        progress: 0,
      ),
    );
  }
}

final class _UploadDefinition {
  const _UploadDefinition({
    required this.projectId,
    required this.ownerId,
    required this.filePath,
  });

  final String projectId;
  final String ownerId;
  final String filePath;
}
```

This supports pause and resume during the active process. True process-death resumability should be added through Android WorkManager and iOS background URL sessions in a later controlled phase.

---

13. Upload Coordinator

`creator_upload_coordinator.dart`

```
import 'dart:async';
```

```dart
import '../data/upload_repository.dart';
import '../domain/creator_project.dart';
import '../domain/creator_project_status.dart';
import '../data/creator_project_repository.dart';

final class CreatorUploadCoordinator {
  CreatorUploadCoordinator({
    required UploadRepository uploads,
    required CreatorProjectRepository projects,
  })  : _uploads = uploads,
        _projects = projects;

  final UploadRepository _uploads;
  final CreatorProjectRepository _projects;

  final Map<String, StreamSubscription<VideoUploadState>>
      _subscriptions = {};

  Future<String> begin(CreatorProject project) async {
    final path = project.renderedPath;

    if (path == null || path.isEmpty) {
      throw StateError('Project has not been rendered.');
    }

    final uploadId = await _uploads.start(
      projectId: project.id,
      ownerId: project.ownerId,
      filePath: path,
    );

    await _projects.save(
      project.copyWith(
        status: CreatorProjectStatus.uploading,
        uploadId: uploadId,
      ),
    );

    _subscriptions[uploadId] =
        _uploads.watch(uploadId).listen(
      (state) async {
        final current =
            await _projects.findById(project.id);

        if (current == null) {
          return;
        }

        switch (state.status) {
          case VideoUploadStatus.completed:
            await _projects.save(
              current.copyWith(
                status: CreatorProjectStatus.uploaded,
                remoteVideoUrl: state.remoteUrl,
              ),
            );

          case VideoUploadStatus.failed:
            await _projects.save(
              current.copyWith(
                status: CreatorProjectStatus.failed,
              ),
            );
```

```
          case VideoUploadStatus.cancelled:
            await _projects.save(
              current.copyWith(
                status: CreatorProjectStatus.rendered,
              ),
            );

          case VideoUploadStatus.paused:
          case VideoUploadStatus.queued:
          case VideoUploadStatus.uploading:
            break;
        }
      },
    );

    return uploadId;
  }

  Future<void> disposeUpload(String uploadId) async {
    await _subscriptions.remove(uploadId)?.cancel();
  }
}
```

---

14. Project Controller

creator_project_controller.dart

```
import 'dart:async';

import 'package:flutter/foundation.dart';
import 'package:uuid/uuid.dart';

import '../data/creator_project_repository.dart';
import '../domain/creator_objective.dart';
import '../domain/creator_project.dart';
import '../domain/creator_project_status.dart';

final class CreatorProjectController extends ChangeNotifier {
  CreatorProjectController({
    required CreatorProjectRepository repository,
    Uuid uuid = const Uuid(),
  })  : _repository = repository,
        _uuid = uuid;

  final CreatorProjectRepository _repository;
  final Uuid _uuid;

  CreatorProject? _project;
  CreatorProject? get project => _project;

  bool _saving = false;
  bool get saving => _saving;

  Timer? _autosaveTimer;

  Future<CreatorProject> create({
    required String ownerId,
    required String title,
    required CreatorObjective objective,
```

```
}) async {
  final now = DateTime.now();

  final project = CreatorProject(
    id: _uuid.v4(),
    ownerId: ownerId,
    title: title,
    objective: objective,
    status: CreatorProjectStatus.draft,
    clips: const [],
    audioTracks: const [],
    captions: const [],
    timelineObjects: const [],
    createdAt: now,
    updatedAt: now,
  );

  _project = project;
  await _repository.save(project);

  notifyListeners();

  return project;
}

Future<void> load(String projectId) async {
  _project = await _repository.findById(projectId);
  notifyListeners();
}

void update(CreatorProject project) {
  _project = project.copyWith(
    updatedAt: DateTime.now(),
  );

  notifyListeners();
  _scheduleAutosave();
}

void _scheduleAutosave() {
  _autosaveTimer?.cancel();

  _autosaveTimer = Timer(
    const Duration(milliseconds: 700),
    save,
  );
}

Future<void> save() async {
  final project = _project;

  if (project == null || _saving) {
    return;
  }

  _saving = true;
  notifyListeners();

  try {
    await _repository.save(project);
  } finally {
    _saving = false;
    notifyListeners();
```

```
    }
  }

  @override
  void dispose() {
    _autosaveTimer?.cancel();
    super.dispose();
  }
}
```

---

15. Camera Controller

creator_camera_controller.dart

```
import 'package:camera/camera.dart';
import 'package:flutter/foundation.dart';
import 'package:uuid/uuid.dart';

import '../domain/video_clip_edit.dart';
import '../services/creator_camera_service.dart';

final class CreatorCameraController extends ChangeNotifier {
  CreatorCameraController({
    required CreatorCameraService cameraService,
    Uuid uuid = const Uuid(),
  })  : _cameraService = cameraService,
        _uuid = uuid;

  final CreatorCameraService _cameraService;
  final Uuid _uuid;

  List<CameraDescription> _cameras = const [];
  List<VideoClipEdit> _clips = const [];

  List<VideoClipEdit> get clips => _clips;
  CameraController? get camera => _cameraService.controller;

  bool _recording = false;
  bool get recording => _recording;

  bool _paused = false;
  bool get paused => _paused;

  Future<void> initialize() async {
    _cameras = await _cameraService.discover();

    if (_cameras.isEmpty) {
      throw StateError('No camera is available.');
    }

    await _cameraService.initialize(
      camera: _cameras.first,
      resolution: ResolutionPreset.high,
    );

    notifyListeners();
  }

  Future<void> start() async {
    await _cameraService.startRecording();
```

```
    _recording = true;
    _paused = false;
    notifyListeners();
  }

  Future<void> pause() async {
    await _cameraService.pauseRecording();
    _paused = true;
    notifyListeners();
  }

  Future<void> resume() async {
    await _cameraService.resumeRecording();
    _paused = false;
    notifyListeners();
  }

  Future<VideoClipEdit> stop({
    required int durationMs,
  }) async {
    final path = await _cameraService.stopRecording();

    final clip = VideoClipEdit(
      id: _uuid.v4(),
      sourcePath: path,
      originalDurationMs: durationMs,
      trimStartMs: 0,
      trimEndMs: durationMs,
    );

    _clips = [..._clips, clip];
    _recording = false;
    _paused = false;

    notifyListeners();

    return clip;
  }

  void deleteLastClip() {
    if (_clips.isEmpty) {
      return;
    }

    _clips = _clips.sublist(0, _clips.length - 1);
    notifyListeners();
  }

  Future<void> switchCamera() async {
    await _cameraService.switchCamera();
    notifyListeners();
  }

  Future<void> setFlash(FlashMode mode) async {
    await _cameraService.setFlashMode(mode);
  }

  @override
  void dispose() {
    _cameraService.dispose();
    super.dispose();
  }
}
```

16. Editor Controller

creator_editor_controller.dart

```
import 'package:flutter/foundation.dart';
import 'package:uuid/uuid.dart';

import '../domain/audio_track_edit.dart';
import '../domain/caption_segment.dart';
import '../domain/creator_project.dart';
import '../domain/timeline_object.dart';
import '../domain/video_clip_edit.dart';

final class CreatorEditorController extends ChangeNotifier {
  CreatorEditorController({
    required CreatorProject initialProject,
    Uuid uuid = const Uuid(),
  })  : _project = initialProject,
        _uuid = uuid;

  final Uuid _uuid;

  CreatorProject _project;
  CreatorProject get project => _project;

  final List<CreatorProject> _undoStack = [];
  final List<CreatorProject> _redoStack = [];

  void _commit(CreatorProject next) {
    _undoStack.add(_project);
    _redoStack.clear();

    _project = next.copyWith(
      updatedAt: DateTime.now(),
    );

    notifyListeners();
  }

  void replaceClips(List<VideoClipEdit> clips) {
    _commit(_project.copyWith(clips: clips));
  }

  void trimClip({
    required String clipId,
    required int startMs,
    required int endMs,
  }) {
    final clips = _project.clips.map((clip) {
      if (clip.id != clipId) {
        return clip;
      }

      if (startMs < 0 ||
          endMs > clip.originalDurationMs ||
          endMs <= startMs) {
        throw ArgumentError('Invalid trim boundaries.');
      }

      return clip.copyWith(
```

```
      trimStartMs: startMs,
      trimEndMs: endMs,
    );
  }).toList();

  _commit(_project.copyWith(clips: clips));
}

void splitClip({
  required String clipId,
  required int splitAtMs,
}) {
  final index = _project.clips.indexWhere(
    (clip) => clip.id == clipId,
  );

  if (index < 0) {
    throw StateError('Clip not found.');
  }

  final clip = _project.clips[index];

  if (splitAtMs <= clip.trimStartMs ||
      splitAtMs >= clip.trimEndMs) {
    throw ArgumentError('Split point is outside the clip.');
  }

  final first = clip.copyWith(
    id: _uuid.v4(),
    trimEndMs: splitAtMs,
  );

  final second = clip.copyWith(
    id: _uuid.v4(),
    trimStartMs: splitAtMs,
  );

  final clips = [..._project.clips]
    ..removeAt(index)
    ..insertAll(index, [first, second]);

  _commit(_project.copyWith(clips: clips));
}

void deleteClip(String clipId) {
  _commit(
    _project.copyWith(
      clips: _project.clips
          .where((clip) => clip.id != clipId)
          .toList(),
    ),
  );
}

void reorderClip(int oldIndex, int newIndex) {
  final clips = [..._project.clips];

  if (newIndex > oldIndex) {
    newIndex -= 1;
  }

  final clip = clips.removeAt(oldIndex);
  clips.insert(newIndex, clip);
```

```
  _commit(_project.copyWith(clips: clips));
}

void updateSpeed(String clipId, double speed) {
  if (speed < 0.25 || speed > 4) {
    throw ArgumentError('Speed must be between 0.25 and 4.');
  }

  _commit(
    _project.copyWith(
      clips: _project.clips.map((clip) {
        return clip.id == clipId
            ? clip.copyWith(speed: speed)
            : clip;
      }).toList(),
    ),
  );
}

void updateVolume(String clipId, double volume) {
  _commit(
    _project.copyWith(
      clips: _project.clips.map((clip) {
        return clip.id == clipId
            ? clip.copyWith(
                volume: volume.clamp(0, 1),
              )
            : clip;
      }).toList(),
    ),
  );
}

void addAudioTrack(AudioTrackEdit track) {
  _commit(
    _project.copyWith(
      audioTracks: [..._project.audioTracks, track],
    ),
  );
}

void setCaptions(List<CaptionSegment> captions) {
  _commit(_project.copyWith(captions: captions));
}

void addTimelineObject(TimelineObject object) {
  if (object.disappearAtMs <= object.appearAtMs) {
    throw ArgumentError('Invalid object visibility period.');
  }

  if (object.disappearAtMs > _project.totalDurationMs) {
    throw ArgumentError(
      'Object exceeds the project duration.',
    );
  }

  _commit(
    _project.copyWith(
      timelineObjects: [
        ..._project.timelineObjects,
        object,
      ],
```

```
      ),
    );
  }

  void removeTimelineObject(String objectId) {
    _commit(
      _project.copyWith(
        timelineObjects: _project.timelineObjects
            .where((object) => object.id != objectId)
            .toList(),
      ),
    );
  }

  void undo() {
    if (_undoStack.isEmpty) {
      return;
    }

    _redoStack.add(_project);
    _project = _undoStack.removeLast();
    notifyListeners();
  }

  void redo() {
    if (_redoStack.isEmpty) {
      return;
    }

    _undoStack.add(_project);
    _project = _redoStack.removeLast();
    notifyListeners();
  }
}
```

---

17. Publish Controller

`creator_publish_controller.dart`

```
import 'package:flutter/foundation.dart';

import '../domain/creator_project.dart';
import '../domain/publication_gate.dart';

final class CreatorPublishController extends ChangeNotifier {
  CreatorPublishController({
    VideoPublicationGate publicationGate =
        const VideoPublicationGate(),
  }) : _publicationGate = publicationGate;

  final VideoPublicationGate _publicationGate;

  PublicationGateResult? _lastResult;
  PublicationGateResult? get lastResult => _lastResult;

  PublicationGateResult evaluate({
    required CreatorProject project,
    required bool moderationPassed,
    required bool copyrightPassed,
    required bool captionReviewComplete,
```

```
    required bool aiChecklistRequired,
    required bool aiChecklistComplete,
  }) {
    final validObjects = project.timelineObjects.every(
      (object) =>
          object.appearAtMs >= 0 &&
          object.disappearAtMs <= project.totalDurationMs &&
          object.deepLink.isNotEmpty,
    );

    _lastResult = _publicationGate.evaluate(
      PublicationGateInput(
        hasRenderedVideo:
            project.renderedPath?.isNotEmpty == true,
        uploadComplete:
            project.remoteVideoUrl?.isNotEmpty == true,
        moderationPassed: moderationPassed,
        copyrightPassed: copyrightPassed,
        metadataComplete:
            project.title.trim().isNotEmpty &&
            project.selectedCaption?.trim().isNotEmpty == true,
        captionReviewComplete: captionReviewComplete,
        validTimelineObjects: validObjects,
        aiChecklistRequired: aiChecklistRequired,
        aiChecklistComplete: aiChecklistComplete,
      ),
    );

    notifyListeners();

    return _lastResult!;
  }
}
```

---

18. Camera Screen

creator_camera_screen.dart

```
import 'dart:async';

import 'package:camera/camera.dart';
import 'package:flutter/material.dart';

import '../controllers/creator_camera_controller.dart';

class CreatorCameraScreen extends StatefulWidget {
  const CreatorCameraScreen({
    super.key,
    required this.controller,
    required this.onComplete,
  });

  final CreatorCameraController controller;
  final void Function(List<VideoClipEdit> clips) onComplete;

  @override
  State<CreatorCameraScreen> createState() =>
      _CreatorCameraScreenState();
}
```

```
class _CreatorCameraScreenState
    extends State<CreatorCameraScreen> {
  final Stopwatch _recordingStopwatch = Stopwatch();
  Timer? _ticker;

  @override
  void initState() {
    super.initState();

    widget.controller.addListener(_refresh);
    widget.controller.initialize();
  }

  void _refresh() {
    if (mounted) {
      setState(() {});
    }
  }

  Future<void> _start() async {
    _recordingStopwatch
      ..reset()
      ..start();

    _ticker = Timer.periodic(
      const Duration(milliseconds: 250),
      (_) => _refresh(),
    );

    await widget.controller.start();
  }

  Future<void> _stop() async {
    _recordingStopwatch.stop();
    _ticker?.cancel();

    await widget.controller.stop(
      durationMs: _recordingStopwatch.elapsedMilliseconds,
    );
  }

  @override
  void dispose() {
    _ticker?.cancel();
    widget.controller
      ..removeListener(_refresh)
      ..dispose();

    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    final camera = widget.controller.camera;

    return Scaffold(
      backgroundColor: Colors.black,
      body: SafeArea(
        child: camera == null || !camera.value.isInitialized
            ? const Center(
                child: CircularProgressIndicator(),
              )
            : Stack(
```

```
fit: StackFit.expand,
children: [
  CameraPreview(camera),
  Align(
    alignment: Alignment.topCenter,
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Text(
        widget.controller.recording
            ? _durationText(
                _recordingStopwatch.elapsed,
              )
            : '${widget.controller.clips.length} clips',
        style: const TextStyle(
          color: Colors.white,
          fontWeight: FontWeight.w700,
        ),
      ),
    ),
  ),
  Align(
    alignment: Alignment.bottomCenter,
    child: Padding(
      padding: const EdgeInsets.all(20),
      child: Row(
        mainAxisAlignment:
            MainAxisAlignment.spaceEvenly,
        children: [
          IconButton.filledTonal(
            tooltip: 'Delete last clip',
            onPressed:
                widget.controller.recording
                    ? null
                    : widget
                        .controller.deleteLastClip,
            icon: const Icon(Icons.undo),
          ),
          GestureDetector(
            onTap: widget.controller.recording
                ? _stop
                : _start,
            child: AnimatedContainer(
              duration:
                  const Duration(milliseconds: 180),
              width: 76,
              height: 76,
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                color: widget.controller.recording
                    ? Colors.red
                    : Colors.white,
                border: Border.all(
                  color: Colors.white,
                  width: 4,
                ),
              ),
            ),
          ),
          IconButton.filledTonal(
            tooltip: 'Switch camera',
            onPressed: widget.controller.recording
                ? null
                : widget.controller.switchCamera,
```

```
                          icon:
                              const Icon(Icons.cameraswitch),
                        ),
                      ],
                    ),
                  ),
                ),
                if (!widget.controller.recording &&
                    widget.controller.clips.isNotEmpty)
                  Positioned(
                    top: 16,
                    right: 16,
                    child: FilledButton(
                      onPressed: () {
                        widget.onComplete(
                          widget.controller.clips,
                        );
                      },
                      child: const Text('Continue'),
                    ),
                  ),
              ],
            ),
      ),
    );
  }

  String _durationText(Duration duration) {
    final minutes =
        duration.inMinutes.remainder(60).toString().padLeft(2, '0');

    final seconds =
        duration.inSeconds.remainder(60).toString().padLeft(2, '0');

    return '$minutes:$seconds';
  }
}
```

Required import:

```
import '../domain/video_clip_edit.dart';
```

---

19. Timeline Widget

`timeline_strip.dart`

```
import 'package:flutter/material.dart';

import '../../domain/video_clip_edit.dart';
import 'clip_tile.dart';

class TimelineStrip extends StatelessWidget {
  const TimelineStrip({
    super.key,
    required this.clips,
    required this.onReorder,
    required this.onDelete,
    required this.onSelect,
    this.selectedClipId,
  });
```

```
  final List<VideoClipEdit> clips;
  final void Function(int oldIndex, int newIndex) onReorder;
  final void Function(String clipId) onDelete;
  final void Function(VideoClipEdit clip) onSelect;
  final String? selectedClipId;

  @override
  Widget build(BuildContext context) {
    return ReorderableListView.builder(
      scrollDirection: Axis.horizontal,
      itemCount: clips.length,
      onReorder: onReorder,
      itemBuilder: (context, index) {
        final clip = clips[index];

        return SizedBox(
          key: ValueKey(clip.id),
          width: 132,
          child: ClipTile(
            clip: clip,
            selected: selectedClipId == clip.id,
            onTap: () => onSelect(clip),
            onDelete: () => onDelete(clip.id),
          ),
        );
      },
    );
  }
}
```

clip_tile.dart

```
import 'package:flutter/material.dart';

import '../../domain/video_clip_edit.dart';

class ClipTile extends StatelessWidget {
  const ClipTile({
    super.key,
    required this.clip,
    required this.selected,
    required this.onTap,
    required this.onDelete,
  });

  final VideoClipEdit clip;
  final bool selected;
  final VoidCallback onTap;
  final VoidCallback onDelete;

  @override
  Widget build(BuildContext context) {
    return Card(
      shape: RoundedRectangleBorder(
        side: BorderSide(
          color: selected
              ? Theme.of(context).colorScheme.primary
              : Colors.transparent,
          width: 2,
        ),
        borderRadius: BorderRadius.circular(14),
```

```
      ),
      child: InkWell(
        borderRadius: BorderRadius.circular(14),
        onTap: onTap,
        child: Padding(
          padding: const EdgeInsets.all(10),
          child: Column(
            children: [
              const Expanded(
                child: Center(
                  child: Icon(Icons.movie_outlined),
                ),
              ),
              Text(
                '${(clip.effectiveDurationMs / 1000).toStringAsFixed(1)}s',
              ),
              IconButton(
                tooltip: 'Delete clip',
                onPressed: onDelete,
                icon: const Icon(Icons.delete_outline),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

---

20. Caption Overlay

caption_overlay.dart

```
import 'package:flutter/material.dart';

import '../../domain/caption_segment.dart';

class CaptionOverlay extends StatelessWidget {
  const CaptionOverlay({
    super.key,
    required this.captions,
    required this.positionMs,
  });

  final List<CaptionSegment> captions;
  final int positionMs;

  @override
  Widget build(BuildContext context) {
    final active = captions.where(
      (caption) =>
          positionMs >= caption.startMs &&
          positionMs <= caption.endMs,
    );

    if (active.isEmpty) {
      return const SizedBox.shrink();
    }

    return SafeArea(
```

```
      child: Align(
        alignment: const Alignment(0, 0.72),
        child: Padding(
          padding: const EdgeInsets.symmetric(horizontal: 24),
          child: DecoratedBox(
            decoration: BoxDecoration(
              color: Colors.black.withValues(alpha: 0.7),
              borderRadius: BorderRadius.circular(12),
            ),
            child: Padding(
              padding: const EdgeInsets.symmetric(
                horizontal: 14,
                vertical: 9,
              ),
              child: Text(
                active.first.text,
                textAlign: TextAlign.center,
                style: const TextStyle(
                  color: Colors.white,
                  fontSize: 18,
                  fontWeight: FontWeight.w700,
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}
```

---

21. Publication Gate Panel

publication_gate_panel.dart

```
import 'package:flutter/material.dart';

import '../../domain/publication_gate.dart';

class PublicationGatePanel extends StatelessWidget {
  const PublicationGatePanel({
    super.key,
    required this.result,
  });

  final PublicationGateResult result;

  @override
  Widget build(BuildContext context) {
    if (result.allowed) {
      return const Card(
        child: ListTile(
          leading: Icon(Icons.verified),
          title: Text('Ready to publish'),
          subtitle: Text(
            'All mandatory publication checks have passed.',
          ),
        ),
      );
    }
```

```
    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            const Text(
              'Publishing blocked',
              style: TextStyle(
                fontWeight: FontWeight.w800,
              ),
            ),
            const SizedBox(height: 10),
            ...result.blockers.map(
              (blocker) => ListTile(
                dense: true,
                leading: const Icon(Icons.error_outline),
                title: Text(blocker),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

---

22. Upload Progress Card

upload_progress_card.dart

```
import 'package:flutter/material.dart';

import '../../data/upload_repository.dart';

class UploadProgressCard extends StatelessWidget {
  const UploadProgressCard({
    super.key,
    required this.state,
    required this.onPause,
    required this.onResume,
    required this.onRetry,
    required this.onCancel,
  });

  final VideoUploadState state;
  final VoidCallback onPause;
  final VoidCallback onResume;
  final VoidCallback onRetry;
  final VoidCallback onCancel;

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            LinearProgressIndicator(
```

```
              value: state.progress.clamp(0, 1),
            ),
            const SizedBox(height: 12),
            Text(
              '${(state.progress * 100).round()}%',
            ),
            const SizedBox(height: 12),
            Wrap(
              spacing: 8,
              children: [
                if (state.status ==
                    VideoUploadStatus.uploading)
                  OutlinedButton(
                    onPressed: onPause,
                    child: const Text('Pause'),
                  ),
                if (state.status ==
                    VideoUploadStatus.paused)
                  FilledButton(
                    onPressed: onResume,
                    child: const Text('Resume'),
                  ),
                if (state.status ==
                    VideoUploadStatus.failed)
                  FilledButton(
                    onPressed: onRetry,
                    child: const Text('Retry'),
                  ),
                TextButton(
                  onPressed: onCancel,
                  child: const Text('Cancel'),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

---

23. Backend Types

creator-studio.types.ts

```
export type TimelineObjectType =
  | "job"
  | "hustle"
  | "marketProduct"
  | "course"
  | "business"
  | "walletOffer"
  | "aiAsk";

export interface TimelineObjectInput {
  id: string;
  type: TimelineObjectType;
  referenceId: string;
  appearAtMs: number;
  disappearAtMs: number;
```

```
}

export interface PublishCreatorVideoInput {
  projectId: string;
  uploadId: string;
  title: string;
  caption: string;
  hashtags: string[];
  visibility: "public" | "followers" | "private";
  captionsReviewed: boolean;
  aiChecklistComplete: boolean;
  timelineObjects: TimelineObjectInput[];
}

export interface PublicationGateState {
  renderComplete: boolean;
  uploadComplete: boolean;
  processingComplete: boolean;
  moderationPassed: boolean;
  copyrightPassed: boolean;
  metadataComplete: boolean;
  captionsReviewed: boolean;
  objectsValid: boolean;
  aiChecklistRequired: boolean;
  aiChecklistComplete: boolean;
}
```

---

24. Backend Publication Gate

publication-gate.ts

```
import type {
  PublicationGateState,
} from "./creator-studio.types";

export interface PublicationDecision {
  allowed: boolean;
  blockers: string[];
}

export function evaluatePublicationGate(
  state: PublicationGateState,
): PublicationDecision {
  const blockers: string[] = [];

  if (!state.renderComplete) {
    blockers.push("Rendering is incomplete.");
  }

  if (!state.uploadComplete) {
    blockers.push("Upload is incomplete.");
  }

  if (!state.processingComplete) {
    blockers.push("Video processing is incomplete.");
  }

  if (!state.moderationPassed) {
    blockers.push("Safety moderation has not passed.");
  }
```

```
  if (!state.copyrightPassed) {
    blockers.push("Copyright verification has not passed.");
  }

  if (!state.metadataComplete) {
    blockers.push("Publishing metadata is incomplete.");
  }

  if (!state.captionsReviewed) {
    blockers.push("Captions require creator review.");
  }

  if (!state.objectsValid) {
    blockers.push("Interactive objects are invalid.");
  }

  if (
    state.aiChecklistRequired &&
    !state.aiChecklistComplete
  ) {
    blockers.push("AI publishing checklist is incomplete.");
  }

  return {
    allowed: blockers.length === 0,
    blockers,
  };
}
```

---

25. Trusted Ecosystem Object Resolver

trusted-object-resolver.ts

```
import {
  getFirestore,
} from "firebase-admin/firestore";

import type {
  TimelineObjectInput,
} from "./creator-studio.types";

const firestore = getFirestore();

export interface ResolvedTimelineObject
  extends TimelineObjectInput {
  title: string;
  deepLink: string;
  ownerId?: string;
  active: boolean;
}

export async function resolveTimelineObject(
  object: TimelineObjectInput,
): Promise<ResolvedTimelineObject> {
  if (
    object.appearAtMs < 0 ||
    object.disappearAtMs <= object.appearAtMs
  ) {
    throw new Error("Invalid object timeline.");
```

```
  }

  switch (object.type) {
    case "job":
      return resolveDocumentObject(
        "jobs",
        object,
        (id) => `yohpal://jobs/${id}`,
      );

    case "hustle":
      return resolveDocumentObject(
        "hustles",
        object,
        (id) => `yohpal://jobs/hustles/${id}`,
      );

    case "marketProduct":
      return resolveDocumentObject(
        "products",
        object,
        (id) => `yohpal://market/products/${id}`,
      );

    case "course":
      return resolveDocumentObject(
        "courses",
        object,
        (id) => `yohpal://courses/${id}`,
      );

    case "business":
      return resolveDocumentObject(
        "businessProfiles",
        object,
        (id) => `yohpal://business/${id}`,
      );

    case "walletOffer":
      return resolveDocumentObject(
        "walletOffers",
        object,
        (id) => `yohpal://wallet/offers/${id}`,
      );

    case "aiAsk":
      return {
        ...object,
        title: "Ask YohPal Brain",
        deepLink:
          `yohpal://brain/ask?context=${encodeURIComponent(object.referenceId)}`,
        active: true,
      };
  }
}

async function resolveDocumentObject(
  collection: string,
  object: TimelineObjectInput,
  createDeepLink: (id: string) => string,
): Promise<ResolvedTimelineObject> {
  const snapshot = await firestore
    .collection(collection)
```

```
    .doc(object.referenceId)
    .get();

  if (!snapshot.exists) {
    throw new Error(
      `${object.type} ${object.referenceId} was not found.`,
    );
  }

  const data = snapshot.data() ?? {};

  const active =
    data.active === true ||
    data.status === "active" ||
    data.status === "published";

  if (!active) {
    throw new Error(
      `${object.type} ${object.referenceId} is not active.`,
    );
  }

  return {
    ...object,
    title:
      String(data.title ?? data.name ?? object.referenceId),
    deepLink: createDeepLink(object.referenceId),
    ownerId:
      typeof data.ownerId === "string"
        ? data.ownerId
        : undefined,
    active: true,
  };
}
```

---

26. Create Upload Session

create-upload-session.ts

```
import {
  onCall,
  HttpsError,
} from "firebase-functions/v2/https";
import {
  FieldValue,
  getFirestore,
} from "firebase-admin/firestore";
import {
  randomUUID,
} from "node:crypto";

const firestore = getFirestore();

export const createCreatorUploadSession = onCall(
  async (request) => {
    const uid = request.auth?.uid;

    if (!uid) {
      throw new HttpsError(
        "unauthenticated",
```

```
        "Authentication is required.",
      );
    }

    const projectId = String(
      request.data?.projectId ?? "",
    );

    if (!projectId) {
      throw new HttpsError(
        "invalid-argument",
        "projectId is required.",
      );
    }

    const uploadId = randomUUID();

    await firestore
      .collection("creatorUploadSessions")
      .doc(uploadId)
      .set({
        uploadId,
        projectId,
        ownerId: uid,
        status: "created",
        createdAt: FieldValue.serverTimestamp(),
        updatedAt: FieldValue.serverTimestamp(),
      });

    return {
      uploadId,
      storagePath:
        `creator-videos/${uid}/${projectId}/${uploadId}.mp4`,
    };
  },
);
```

---

27. Finalize Upload

finalize-video-upload.ts

```
import {
  onCall,
  HttpsError,
} from "firebase-functions/v2/https";
import {
  FieldValue,
  getFirestore,
} from "firebase-admin/firestore";
import {
  getStorage,
} from "firebase-admin/storage";

const firestore = getFirestore();
const storage = getStorage();

export const finalizeCreatorVideoUpload = onCall(
  async (request) => {
    const uid = request.auth?.uid;
```

```
if (!uid) {
  throw new HttpsError(
    "unauthenticated",
    "Authentication is required.",
  );
}

const uploadId = String(
  request.data?.uploadId ?? "",
);

if (!uploadId) {
  throw new HttpsError(
    "invalid-argument",
    "uploadId is required.",
  );
}

const sessionRef = firestore
  .collection("creatorUploadSessions")
  .doc(uploadId);

const session = await sessionRef.get();

if (!session.exists) {
  throw new HttpsError(
    "not-found",
    "Upload session was not found.",
  );
}

const data = session.data()!;

if (data.ownerId !== uid) {
  throw new HttpsError(
    "permission-denied",
    "This upload belongs to another user.",
  );
}

const storagePath =
  `creator-videos/${uid}/${data.projectId}/${uploadId}.mp4`;

const [exists] = await storage
  .bucket()
  .file(storagePath)
  .exists();

if (!exists) {
  throw new HttpsError(
    "failed-precondition",
    "Uploaded media was not found.",
  );
}

await sessionRef.update({
  status: "uploaded",
  storagePath,
  updatedAt: FieldValue.serverTimestamp(),
});

await firestore
  .collection("creatorProcessingJobs")
```

```
      .doc(uploadId)
      .set({
        uploadId,
        projectId: data.projectId,
        ownerId: uid,
        sourcePath: storagePath,
        operation: "video.transcode-and-moderate",
        status: "queued",
        createdAt: FieldValue.serverTimestamp(),
      });

    return {
      uploadId,
      status: "processing",
    };
  },
);
```

---

28. Publish Video

publish-creator-video.ts

```
import {
  onCall,
  HttpsError,
} from "firebase-functions/v2/https";
import {
  FieldValue,
  getFirestore,
} from "firebase-admin/firestore";

import {
  evaluatePublicationGate,
} from "./publication-gate";
import {
  resolveTimelineObject,
} from "./trusted-object-resolver";
import type {
  PublishCreatorVideoInput,
} from "./creator-studio.types";

const firestore = getFirestore();

export const publishCreatorVideo = onCall(
  async (request) => {
    const uid = request.auth?.uid;

    if (!uid) {
      throw new HttpsError(
        "unauthenticated",
        "Authentication is required.",
      );
    }

    const input =
      request.data as PublishCreatorVideoInput;

    if (
      !input?.projectId ||
      !input?.uploadId ||
```

```
  !input?.title ||
  !input?.caption
) {
  throw new HttpsError(
    "invalid-argument",
    "Project, upload, title and caption are required.",
  );
}

const upload = await firestore
  .collection("creatorUploadSessions")
  .doc(input.uploadId)
  .get();

if (!upload.exists) {
  throw new HttpsError(
    "not-found",
    "Upload session was not found.",
  );
}

const uploadData = upload.data()!;

if (uploadData.ownerId !== uid) {
  throw new HttpsError(
    "permission-denied",
    "This upload belongs to another user.",
  );
}

const processing = await firestore
  .collection("creatorProcessingJobs")
  .doc(input.uploadId)
  .get();

const processingData =
  processing.data() ?? {};

const resolvedObjects = await Promise.all(
  (input.timelineObjects ?? []).map(
    resolveTimelineObject,
  ),
);

const durationMs = Number(
  processingData.durationMs ?? 0,
);

const objectsValid = resolvedObjects.every(
  (object) =>
    object.appearAtMs >= 0 &&
    object.disappearAtMs <= durationMs &&
    object.active,
);

const gate = evaluatePublicationGate({
  renderComplete: true,
  uploadComplete:
    uploadData.status === "uploaded",
  processingComplete:
    processingData.status === "completed",
  moderationPassed:
    processingData.moderationStatus === "approved",
```

```
  copyrightPassed:
    processingData.copyrightStatus === "approved",
  metadataComplete:
    input.title.trim().length > 0 &&
    input.caption.trim().length > 0,
  captionsReviewed: input.captionsReviewed,
  objectsValid,
  aiChecklistRequired:
    process.env.CREATOR_AI_GATE_REQUIRED === "true",
  aiChecklistComplete:
    input.aiChecklistComplete === true,
});

if (!gate.allowed) {
  throw new HttpsError(
    "failed-precondition",
    gate.blockers.join(" "),
    {
      blockers: gate.blockers,
    },
  );
}

const videoRef =
  firestore.collection("videos").doc();

await firestore.runTransaction(
  async (transaction) => {
    transaction.set(videoRef, {
      id: videoRef.id,
      ownerId: uid,
      userId: uid,
      projectId: input.projectId,
      uploadId: input.uploadId,
      title: input.title.trim(),
      caption: input.caption.trim(),
      hashtags: input.hashtags ?? [],
      visibility: input.visibility,
      status: "published",
      hlsUrl: processingData.hlsUrl,
      videoUrl: processingData.videoUrl,
      thumbnailUrl:
        processingData.thumbnailUrl ?? null,
      previewUrl:
        processingData.previewUrl ?? null,
      durationMs,
      interactiveObjects: resolvedObjects,
      views: 0,
      likes: 0,
      shares: 0,
      comments: 0,
      createdAt: FieldValue.serverTimestamp(),
      updatedAt: FieldValue.serverTimestamp(),
    });

    transaction.update(upload.ref, {
      status: "published",
      videoId: videoRef.id,
      updatedAt: FieldValue.serverTimestamp(),
    });
  },
);
```

```
    return {
      published: true,
      videoId: videoRef.id,
    };
  },
);
```

---

29. Function Exports

creator-studio.routes.ts

```
export {
  createCreatorUploadSession,
} from "./create-upload-session";

export {
  finalizeCreatorVideoUpload,
} from "./finalize-video-upload";

export {
  publishCreatorVideo,
} from "./publish-creator-video";
```

Add to the main Firebase Functions entry file:

```
export * from "./creator-studio/creator-studio.routes";
```

---

30. Firestore Rules

firestore/firestore.rules

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    function signedIn() {
      return request.auth != null;
    }

    function owns(resourceData) {
      return signedIn()
        && resourceData.ownerId == request.auth.uid;
    }

    match /creatorProjects/{projectId} {
      allow read: if signedIn()
        && resource.data.ownerId == request.auth.uid;

      allow create: if signedIn()
        && request.resource.data.ownerId == request.auth.uid;

      allow update: if owns(resource.data)
        && request.resource.data.ownerId == resource.data.ownerId;

      allow delete: if owns(resource.data);
    }
```

```
    match /creatorUploadSessions/{uploadId} {
      allow read: if signedIn()
        && resource.data.ownerId == request.auth.uid;

      allow create, update, delete: if false;
    }

    match /creatorProcessingJobs/{jobId} {
      allow read: if signedIn()
        && resource.data.ownerId == request.auth.uid;

      allow create, update, delete: if false;
    }

    match /videos/{videoId} {
      allow read: if resource.data.status == "published"
        && resource.data.visibility == "public";

      allow create, update, delete: if false;
    }

    match /creatorAiJobs/{jobId} {
      allow read: if signedIn()
        && resource.data.creatorId == request.auth.uid;

      allow create: if signedIn()
        && request.resource.data.creatorId == request.auth.uid;

      allow update, delete: if false;
    }
  }
}
```

---

31. Storage Rules

```
rules_version = '2';

service firebase.storage {
  match /b/{bucket}/o {
    match /creator-videos/{userId}/{projectId}/{uploadFile} {
      allow read: if request.auth != null
        && request.auth.uid == userId;

      allow write: if request.auth != null
        && request.auth.uid == userId
        && request.resource.size < 500 * 1024 * 1024
        && request.resource.contentType.matches('video/.*');
    }

    match /creator-audio/{userId}/{projectId}/{audioFile} {
      allow read, write: if request.auth != null
        && request.auth.uid == userId
        && request.resource.size < 100 * 1024 * 1024;
    }

    match /published-videos/{allPaths=**} {
      allow read: if true;
      allow write: if false;
    }
  }
}
```

32. Core Tests

publication_gate_test.dart

```
import 'package:flutter_test/flutter_test.dart';

void main() {
  const gate = VideoPublicationGate();

  test('blocks publishing where moderation has not passed', () {
    final result = gate.evaluate(
      const PublicationGateInput(
        hasRenderedVideo: true,
        uploadComplete: true,
        moderationPassed: false,
        copyrightPassed: true,
        metadataComplete: true,
        captionReviewComplete: true,
        validTimelineObjects: true,
        aiChecklistRequired: false,
        aiChecklistComplete: false,
      ),
    );

    expect(result.allowed, isFalse);
    expect(
      result.blockers,
      contains('Safety moderation has not passed.'),
    );
  });

  test('allows publishing when all mandatory gates pass', () {
    final result = gate.evaluate(
      const PublicationGateInput(
        hasRenderedVideo: true,
        uploadComplete: true,
        moderationPassed: true,
        copyrightPassed: true,
        metadataComplete: true,
        captionReviewComplete: true,
        validTimelineObjects: true,
        aiChecklistRequired: true,
        aiChecklistComplete: true,
      ),
    );

    expect(result.allowed, isTrue);
    expect(result.blockers, isEmpty);
  });
}
```

Add imports matching the project package.

creator_editor_controller_test.dart

```
import 'package:flutter_test/flutter_test.dart';

void main() {
  test('splits clip into two clips', () {
```

```
  final project = CreatorProject(
    id: 'project-1',
    ownerId: 'user-1',
    title: 'Test',
    objective: CreatorObjective.entertain,
    status: CreatorProjectStatus.editing,
    clips: const [
      VideoClipEdit(
        id: 'clip-1',
        sourcePath: '/tmp/video.mp4',
        originalDurationMs: 10000,
        trimStartMs: 0,
        trimEndMs: 10000,
      ),
    ],
    audioTracks: const [],
    captions: const [],
    timelineObjects: const [],
    createdAt: DateTime(2026),
    updatedAt: DateTime(2026),
  );

  final controller = CreatorEditorController(
    initialProject: project,
  );

  controller.splitClip(
    clipId: 'clip-1',
    splitAtMs: 4000,
  );

  expect(controller.project.clips, hasLength(2));
  expect(
    controller.project.clips.first.trimEndMs,
    4000,
  );
  expect(
    controller.project.clips.last.trimStartMs,
    4000,
  );
});

test('reorders clips', () {
  final project = CreatorProject(
    id: 'project-1',
    ownerId: 'user-1',
    title: 'Test',
    objective: CreatorObjective.entertain,
    status: CreatorProjectStatus.editing,
    clips: const [
      VideoClipEdit(
        id: 'a',
        sourcePath: 'a.mp4',
        originalDurationMs: 1000,
        trimStartMs: 0,
        trimEndMs: 1000,
      ),
      VideoClipEdit(
        id: 'b',
        sourcePath: 'b.mp4',
        originalDurationMs: 1000,
        trimStartMs: 0,
        trimEndMs: 1000,
```

```
        ),
      ],
      audioTracks: const [],
      captions: const [],
      timelineObjects: const [],
      createdAt: DateTime(2026),
      updatedAt: DateTime(2026),
    );

    final controller = CreatorEditorController(
      initialProject: project,
    );

    controller.reorderClip(0, 2);

    expect(controller.project.clips.first.id, 'b');
    expect(controller.project.clips.last.id, 'a');
  });
}
```

ffmpeg_command_builder_test.dart

```
import 'package:flutter_test/flutter_test.dart';

void main() {
  test('creates concatenation render plan', () {
    final project = CreatorProject(
      id: 'project',
      ownerId: 'owner',
      title: 'Test',
      objective: CreatorObjective.entertain,
      status: CreatorProjectStatus.editing,
      clips: const [
        VideoClipEdit(
          id: '1',
          sourcePath: '/clips/one.mp4',
          originalDurationMs: 5000,
          trimStartMs: 500,
          trimEndMs: 4500,
        ),
        VideoClipEdit(
          id: '2',
          sourcePath: '/clips/two.mp4',
          originalDurationMs: 6000,
          trimStartMs: 0,
          trimEndMs: 5000,
        ),
      ],
      audioTracks: const [],
      captions: const [],
      timelineObjects: const [],
      createdAt: DateTime(2026),
      updatedAt: DateTime(2026),
    );

    final plan = const FfmpegCommandBuilder().build(
      project: project,
      outputPath: '/output/final.mp4',
    );

    expect(plan.arguments, contains('-filter_complex'));
    expect(plan.arguments.join(' '), contains('concat=n=2'));
```

```
    expect(plan.outputPath, '/output/final.mp4');
  });
}
```

---

33. Validation Script

scripts/validate_creator_studio_1_1a.sh

```
#!/usr/bin/env bash
set -euo pipefail

MOBILE_ROOT="apps/mobile_flutter"
FUNCTIONS_ROOT="backend/firebase_functions"
EVIDENCE_ROOT="release-evidence/creator-studio-1.1a"

mkdir -p "${EVIDENCE_ROOT}"

pushd "${MOBILE_ROOT}" >/dev/null

flutter clean

flutter pub get \
  2>&1 | tee "../../${EVIDENCE_ROOT}/flutter-pub-get.txt"

dart format \
  --output=none \
  --set-exit-if-changed \
  lib test \
  2>&1 | tee "../../${EVIDENCE_ROOT}/dart-format.txt"

flutter analyze \
  2>&1 | tee "../../${EVIDENCE_ROOT}/flutter-analyze.txt"

flutter test \
  2>&1 | tee "../../${EVIDENCE_ROOT}/flutter-test.txt"

popd >/dev/null

pushd "${FUNCTIONS_ROOT}" >/dev/null

npm ci \
  2>&1 | tee "../../${EVIDENCE_ROOT}/functions-npm-ci.txt"

npm run lint \
  2>&1 | tee "../../${EVIDENCE_ROOT}/functions-lint.txt"

npm run build \
  2>&1 | tee "../../${EVIDENCE_ROOT}/functions-build.txt"

npm test \
  2>&1 | tee "../../${EVIDENCE_ROOT}/functions-test.txt"

popd >/dev/null

git branch --show-current \
  > "${EVIDENCE_ROOT}/branch.txt"

git rev-parse HEAD \
  > "${EVIDENCE_ROOT}/commit.txt"
```

```
git status --short \
  > "${EVIDENCE_ROOT}/git-status.txt"

echo "Creator Studio 1.1A validation completed."
```

---

34. Phased Launch Activation

Phase 1 — Launch Foundation

Enable:

```
Creator Studio projects
Gallery imports
Camera recording
Multi-clip recording
Trim
Split
Delete
Reorder
Draft autosave
Upload progress
Cancel and retry
Publication gates
```

Flags:

```
--dart-define=CREATOR_STUDIO_ENABLED=true
--dart-define=CREATOR_CAMERA_ENABLED=true
--dart-define=CREATOR_MULTI_CLIP_ENABLED=true
--dart-define=CREATOR_CAPTIONS_ENABLED=false
--dart-define=CREATOR_MUSIC_ENABLED=false
--dart-define=CREATOR_OBJECTS_ENABLED=false
```

Phase 2 — Caption and Audio Pilot

Enable:

```
Synchronized captions
Caption editing
Caption translation
Music tracks
Voice-over
Audio mixing
```

Phase 3 — Ecosystem Objects

Enable:

```
Jobs
Hustles
Market products
Courses
Business profiles
Wallet offers
AI Ask
```

Phase 4 — AI Production Integration

Enable:

```
AI captions
Hooks
Hashtags
Thumbnail concepts
Viral-readiness score
Trim recommendations
AI-assisted application of edits
```

Phase 5 — Controlled Public Rollout

Enable after:

- Android device validation.
- iPhone validation.
- Rendering stability.
- Upload recovery validation.
- Moderation and copyright evidence.
- Object deep-link tests.
- Wallet and commerce security review.
- Crashlytics evidence.
- Release Review Board approval.

---

35. Release Acceptance Criteria

Gate	Required outcome
Camera recording	PASS
Multi-clip recording	PASS
Trim and split	PASS
Reorder and delete	PASS
Draft autosave	PASS
Undo and redo	PASS
Rendering	PASS
Music and voice-over	PASS

Synchronized captions	PASS
Resumable active upload	PASS
Cancel and retry	PASS
Publication gate	PASS
Trusted timeline objects	PASS
YohPal Brain integration	PASS
Android physical device	PASS
iPhone physical device	PASS
Flutter analysis	PASS
Flutter tests	PASS
Backend build	PASS
Firestore rules	PASS
Storage rules	PASS
Crashlytics	PASS

Current Implementation Decision

Creator Studio Release 1.1A codebase: Approved as the phased launch foundation.

Immediate public activation of every feature: Not recommended.

The safest order is:

1. Camera and core timeline.
2. Drafts and rendering.
3. Reliable upload.
4. Captions and audio.
5. Interactive ecosystem objects.
6. YohPal Brain production integration.
7. Controlled creator pilot.
8. Wider rollout.

The mobile FFmpeg execution adapter must use the FFmpeg runtime already licensed and approved for the YohPal repository. The command builder and render contracts above deliberately isolate that runtime so it can be replaced without rewriting Creator Studio.

Next Smart Move

```
BUILD: YOHPAL LIVE CREATOR STUDIO RELEASE 1.1A-RC1 — CORE CAMERA, TIMELINE & DRAFT PILOT

Activate only:

1. YohPal Camera.
2. Multi-clip recording.
3. Gallery import.
4. Trim.
5. Split.
6. Delete.
7. Reorder.
8. Undo and redo.
9. Draft autosave.
10. Local rendering.
11. Upload progress, cancel and retry.
12. Mandatory publication gates.

Keep captions, music, voice-over, ecosystem objects and AI edit execution behind feature
flags until the core camera-to-render-to-upload flow passes physical Android and iPhone
validation.

Required evidence:

- Immutable branch, commit and tag
- Android camera proof
- iPhone camera proof
- Multi-clip timeline proof
- Trim and split proof
- Draft recovery proof
- Rendered output proof
- Upload cancellation and retry proof
- Publication blocking proof
- flutter analyze
- flutter test
- Backend build and tests
- Crashlytics evidence

Outcome:

YohPal launches Creator Studio progressively without exposing unverified advanced features
or delaying the production release.
```